

Maybe a bit. But don't assume all the systems in the world have huge amounts of resources available. Consider embedded systems, for example. The microchips in our washing machines and cars do not have huge memories or powerful processors. So embedded system programming still involves a lot of assembly coding.

Third, learning assembly language programming will enhance your knowledge of computers in general. A high-level language programmer can write thousands of lines of code without caring the least about the underlying architecture of the hardware. Of course, this is really a nice feature of high-level languages. At the same time, if you are a serious programmer, it is a good idea to know what is happening inside the machine when the code you write is executed. If you learn assembly language programming, things will change. You need to have a clear understanding about registers, primary memory, processors, etc, to write a decent assembly language program.

According to Wikipedia, "A program written in assembly language consists of a series of mnemonic processor instructions and meta-statements (known variously as directives, pseudo-instructions, and pseudo-ops), comments and data." Now, before we proceed any further, I would like to clarify an important point. The right term is not 'assembly language' but 'assembly languages', as there are a number of programming languages that fall under this category. Every unique computer architecture has its own assembly language associated with it. An easy classification would be based on the CISC vs RISC classification of architectures. CISC (complex instruction set computer) tries to complete a task in as few lines of assembly code as possible,

whereas RISC (reduced instruction set computer) processors generally use simple instructions that can be executed within one clock cycle. Assembly languages like the MIPS assembly language, ARM assembly language, etc, are associated with the RISC architecture. The x86 assembly language is generally considered as a CISC based assembly language. But this classification of assembly languages is becoming less meaningful, as more and more architectures tend to be a hybrid of both CISC and RISC.

With most laptops having

shown below is executed using this assembler.

```
MOV r0, #15 ;
MOV r1, #10 ;
ADD r2, r0, r1 ;
```

The program stores the decimal value 15 (*f* in hexadecimal) in Register 0 and the decimal value 10 (*a* in hexadecimal) in Register 1; it adds the two numbers and stores the result in Register 2. Figure 1 shows the execution window of OakSim. Figure 2 shows the values in the registers after the execution of the

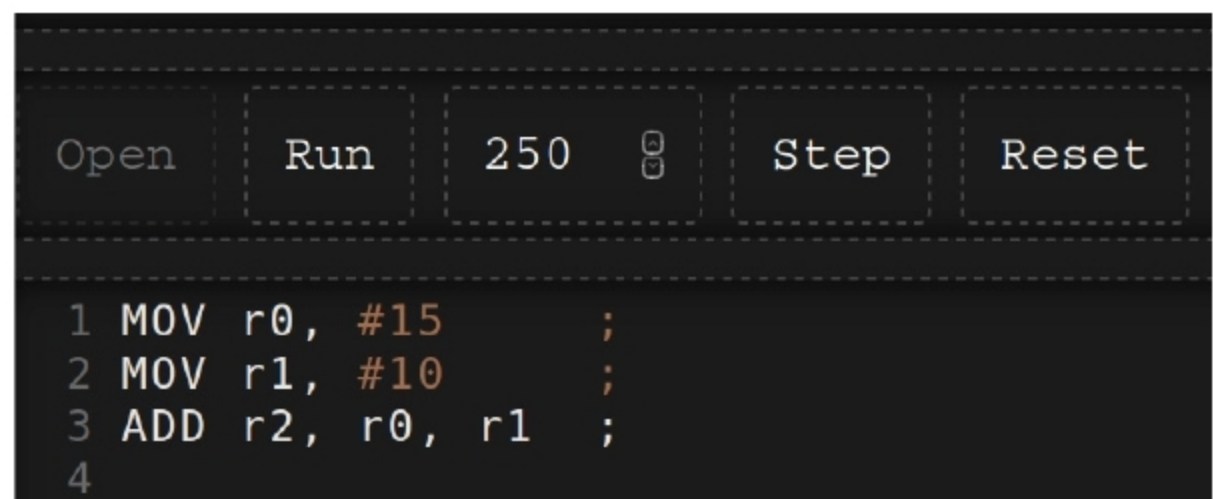


Figure 1: Execution window of OakSim

Intel or AMD processors (both of which use the x86 assembly language), how can we test a RISC based assembly language? I urge you to use Raspberry Pi, a small single-board computer. It uses the ARM assembly language. But I didn't have a Raspberry Pi computer with me while writing this article. So I either had to depend on an ARM emulator or an online ARM assembler. There is an ARM emulator called VisUAL, which can be installed in your laptop to test the ARM assembly language program intended for your Raspberry Pi computer. But I chose to use an online ARM assembler called OakSim. The URL <https://wunkolo.github.io/OakSim/> will take you to the OakSim online assembler. The ARM assembly code

Register Value	
R0	f
R1	a
R2	19
R3	0

Figure 2: Values in the Registers after program execution

program. It can be seen that Register 2 has hexadecimal value 19 (25 in decimal), the sum of the values in Registers 0 and 1.

Now that we have discussed an example for a RISC based assembly language, let us discuss a CISC based assembly language — the x86 assembly language. There are